

Checklist

Platform Engineering Team

Team Composition

- Build a **mixed team** rather than staffing only software engineers or only systems engineers.
- Ensure the team has people who can **write substantial production code**.
- Ensure the team has people with **broad systems and operational expertise**.
- Include reliability expertise where reliability is a significant organizational need.
- Add systems specialists only where deep expertise is genuinely required.
- Avoid overloading the team with specialists at the expense of broad systems engineers.
- Make sure specialists can contribute beyond their specialty when needed.
- Treat software and systems work as **equally valuable** to the platform's success.

Avoid a “Too Much Systems” Culture

- Check that the team is building platform abstractions, not just automation, templates, and one-off tools.
- Invest in architecture that permanently reduces operational complexity.
- Avoid relying on rules, documentation, and manual processes when engineering could remove the problem.
- Do not use technical hiring filters that unintentionally exclude strong software engineers.
- Make sure experienced systems engineers can mentor engineers who are newer to infrastructure.
- Watch for a culture that blames users or developers rather than improving the platform.

Avoid a “Too Much Development” Culture

- Treat maintenance and improvement of existing systems as valuable engineering work.
- Avoid assuming technical debt is simply “someone else's code.”

- Do not prioritize new architecture over reliability without a clear business reason.
- Require engineers to understand how their systems operate in production.
- Make operational responsibility part of engineering expectations.
- Ensure engineers can respond effectively to incidents and production problems.
- Balance experimentation and new development with stability and long-term support.

Software Engineer Expectations

- Hire software engineers who are curious about **systems**, not only application code.
- Look for engineers who want to understand the full environment their code runs in.
- Confirm candidates are comfortable investigating operating systems, networking, storage, databases, distributed systems, or related layers when necessary.
- Expect platform software engineers to participate in operating business-critical systems.
- Confirm engineers are willing and able to participate in on-call responsibilities when appropriate.
- Look for evidence that candidates have meaningfully helped resolve incidents.
- Prefer engineers comfortable shipping at a **deliberate, reliability-conscious pace**.
- Reward work on operations, integrations, experimentation, and support—not only feature code.

Systems Engineer Expectations

- Hire broad systems generalists who understand how multiple parts of the platform interact.
- Look for experience in areas such as infrastructure integration, scaling, reliability, and observability.
- Expect systems engineers to use their knowledge to solve problems across the platform codebase.
- Avoid treating them only as operational debuggers.
- Create a career path that allows broad systems engineers to remain broad rather than forcing specialization.
- Recognize systems engineers for their impact even when that impact produces less new code.

Reliability Engineers

- Use reliability engineering as a **specialized role**, not a catch-all label for all systems work.
- Hire people motivated by improving reliability across teams and systems.
- Look for experience in incident management, SLOs, capacity, production readiness, and operational improvement.
- Ensure reliability engineers have enough technical depth to implement solutions, not only recommend them.
- Give reliability engineers a clear mandate and sufficient organizational reach.
- Avoid isolating reliability engineers in ways that cause their work to lose credibility with platform teams.
- Consider rotating reliability specialists into and out of platform teams to keep their skills current.

Systems Specialists

- Hire specialists only when the organization has a clear need for deep expertise.
- Possible specialties include networking, kernels, performance, and storage.
- Limit the number of specialists until their value is demonstrated.
- Keep specialists close to the teams and systems where their expertise is actually useful.
- Expect specialists to understand the broader platform context.
- Avoid overspecialization that causes engineers to ignore the larger customer or business need.
- Do not create “internal evangelist” roles that are disconnected from practical delivery.

Job Titles and Career Levels

- Allow role-specific titles when they accurately describe meaningful differences in work.
- Do not force everyone into the same title merely for administrative consistency.
- Separate **job title**, **level matrix**, and **interview process** decisions.
- Avoid creating a separate level matrix for every engineering specialty.
- Keep software-development roles on a shared ladder where practical.
- Create no more than one additional systems-oriented level matrix if the existing ladder cannot fairly evaluate systems work.
- Evaluate impact based on outcomes rather than lines of code produced.

- Ensure promotion criteria recognize operational, reliability, systems, and customer impact.
- Use reviewers outside the immediate platform team when useful to demonstrate company-wide impact.

Evidence for Promotion and Recognition

- Document tools, dashboards, or internal documentation that are widely adopted.
- Capture evidence of high-quality customer interactions.
- Track effective handling and resolution of support tickets.
- Record meaningful participation in incident response and postmortems.
- Capture examples of coaching other teams during incidents.
- Document improvements in reliability, operability, scalability, or usability.
- Ensure performance reviews recognize work that reduces future operational burden.

Platform Engineering Interviews

- Customize interviews to evaluate the actual work platform engineers perform.
- Include a coding exercise that tests practical implementation ability.
- Use coding questions that allow discussion of testing, error handling, observability, scaling, and operational constraints.
- Include a systems-oriented coding or technical exercise.
- Include a platform design interview rather than only an application design interview.
- Include an “inverted” design interview where the candidate explains a real system they designed.
- Ask candidates to explain technical tradeoffs in their previous work.
- Include a behavioral interview focused on operational experience.
- Assess whether candidates can remain effective during incidents and ambiguity.
- Evaluate customer empathy as part of the process.
- Train and calibrate interviewers before rolling out a new interview format.
- Collect interviewer feedback and improve the process after early rounds.

Interviews for Systems Roles

- Keep the core interview process similar to the platform software engineering process.
- Allow more flexibility in design questions based on the candidate’s specialty.

- Use the inverted design interview to explore deep systems knowledge.
- Continue evaluating coding ability rather than eliminating coding entirely.
- Consider a time-boxed take-home coding exercise when live coding creates an unrealistic evaluation environment.
- Review the take-home solution during the interview to explore implementation choices and systems thinking.
- Adjust technical depth to the role without creating completely separate hiring standards.

Customer Empathy

- Screen candidates for their ability to understand and respect platform users.
- Ask for examples of helping users understand a system.
- Ask for examples where customer feedback changed what the candidate built.
- Ask how candidates learn what users actually need.
- Look for engineers who can handle frustrated users without becoming dismissive.
- Reject cultures that label users as the problem instead of learning from their experience.
- Reinforce that internal users are still **customers of the platform**.
- Reward engineers for making the platform easier for other people to use.

Platform Engineering Managers

- Hire managers with experience operating complex or business-critical systems.
- Confirm they understand that platform problems often have ambiguous boundaries and causes.
- Look for leaders who solve operational problems rather than dismissing them as temporary noise.
- Avoid leaders who assume every problem can be solved with a simple software rewrite.
- Prefer managers with experience leading large, long-running projects.
- Make sure managers are comfortable with a slower, more deliberate delivery pace when reliability requires it.
- Look for leaders who can defend the team's delivery choices using business criticality, complexity, and risk.
- Evaluate candidates for strong attention to detail.
- Look for managers who enjoy project and process management rather than avoiding it.
- Ensure managers know when to inspect details and when to trust the team.

Product Management

- Treat the platform as a **product**, with customers whose needs must shape priorities.
- Add dedicated product management when the platform reaches sufficient scale.
- Hire PMs who understand internal platforms and engineering customers.
- Avoid PMs who apply external-product assumptions without adapting to platform work.
- Balance short-term business pressure with the long-term health of the platform.
- Recruit product-minded engineers when formal PM hiring is not possible.
- Empower staff engineers with strong customer communication skills to help fill product gaps.
- Pair inexperienced platform PMs with experienced product leaders for coaching.
- Keep strategic product decisions and technical execution clearly coordinated.

Product Owners and TPMs

- Avoid splitting product manager and product owner responsibilities unless there is a clear benefit.
- Make sure someone owns both strategy and execution mechanics.
- Use technical program managers when coordination complexity genuinely requires them.
- Do not use TPMs as a substitute for product strategy or engineering leadership.
- Prefer smaller projects to be run by engineering managers and technical leads where practical.
- Hire TPMs who can succeed within the organization's existing working model.
- Look for TPMs who can build influence without formal authority.
- At larger companies, value TPMs who can communicate effectively with senior leadership.

Supporting Roles

- Add developer advocates when developer communication becomes a full-time need.
- Add technical writers when documentation demands justify specialization.
- Add support engineers when support volume requires dedicated ownership.
- Avoid creating specialist roles prematurely.
- Ensure engineers continue to contribute to documentation, support, and customer communication even when specialists exist.

- Recognize and reward these activities in performance evaluations.

Creating a Balanced Platform Culture

- Explicitly combine **software-building** and **systems-operating** mindsets.
- Balance people who enjoy creating new things with people who enjoy scaling and improving existing things.
- Ensure some team members are strongly oriented toward customer collaboration.
- Maintain room for experimentation without sacrificing stability.
- Make reliability, scalability, and usability team-wide responsibilities.
- Use a shared roadmap rather than allowing every engineer to independently choose features.
- Consolidate prioritization across development and operational work.
- Avoid “software vs. operations” or “us vs. them” thinking.
- Make technical leads responsible for reinforcing collaboration across disciplines.
- Reinforce the platform engineering culture during organizational changes.

Culture and Recognition

- Identify cultural differences between platform teams and surrounding engineering teams.
- Preserve useful company values such as innovation and collaboration.
- Add platform-specific emphasis on stability, reliability, usability, and scale.
- Recognize and reward different engineering skill sets equally.
- Publicly appreciate partner teams rather than blaming them for platform problems.
- Watch for cultural drift that creates contempt between platform and application teams.
- Encourage engineers to see partner teams as customers and collaborators.
- Make culture an explicit leadership responsibility rather than assuming it will emerge automatically.

Final Health Check

- Can the team **build substantial software**, not just configure infrastructure?
- Can the team **operate and debug complex systems**?
- Does the team understand what its customers need?
- Are reliability and usability treated as product requirements?

- Are both software and systems contributions rewarded?
- Can engineers advance their careers without becoming feature-focused developers?
- Are interview processes aligned with actual platform work?
- Do managers understand operational complexity?
- Is product management influencing priorities and customer outcomes?
- Is the team avoiding both “build everything new” and “just keep it running” extremes?
- Does the organization reward collaborative behavior across engineering disciplines?
- Is the platform team staffed to build a **real platform product**, rather than a collection of scripts and glue?